What is Pagination?

When an API needs to return a large list of data (like 10,000 items), sending all at once is not efficient.

Default Behavior (No Pagination):
```json
{
  "data": [item1, item2, ..., item10000]
}
```

Problems:
1. Heavy on Database: Takes time and resources to fetch 10,000 records.
2. Slow API Response: Large data = slow network transfer.
3. Poor User Experience: Users don't need all data at once.

Pagination

Pagination means splitting data into small chunks (pages) and sending one page at a time.

Example:

- Total items: 10,000
- Page size: 50
- Total pages: 200
- User first sees Page 1 → 50 items
- They can go to Page 2, 3, ... as needed

Most users only check the first few pages (like Google search).

Data Size vs. Request Count

Strategy	Effect

Smaller page size	More API calls needed to see all data
Larger page size	Fewer calls, but each response is heavier

Types of Pagination

1)Offset-Based Pagination (Simple & Common)

Uses:

- page and page_size, OR
- offset and limit

How it works:

GET /comments?page=3&size=20

Backend calculates:

offset = (page - 1) * page_size # (3-1)*20 = 40

SQL Query:

SELECT * FROM comments
ORDER BY id
LIMIT 20 OFFSET 40;
This returns items 41 to 60.

Pros:

- Simple to understand and implement.

Cons:

- Data shifts if items are added/deleted.
- Duplicates or skips can happen during navigation.

Example:

Before:
1 2 3 4 5 6 7 8 9 10 → Page 1: 1-5, Page 2: 6-10
Delete 6 and 7:
1 2 3 4 5 8 9 10
Now Page 2: 8 9 10 → Items 6 and 7 are gone, and 8 jumps in.
Not reliable for real-time or frequently changing data.

2)Cursor-Based Pagination (Advanced & Reliable)

Uses a pointer (cursor) to the last seen item.

How it works:

GET /comments?after_id=123

- after_id=123 means: "Give me items after the one with ID 123"
- No offset — just start from a known point

SQL Query:

```
SELECT * FROM comments
WHERE id > 123
ORDER BY id
LIMIT 20;
```

Pros:

- No data skipping/duplication even if items are added/deleted.
- Consistent experience — always moves forward from last seen item.

Limitations:

- Cursor field must be:
    - Unique
    - Not null
    - Never changes
    - Indexed (for fast lookup)
- Usually uses id or created_at.

Response includes next cursor:

```
{
  "data": [...],
```

```
  "next": "/comments?after_id=145"
}
```

Implementation

Page schema

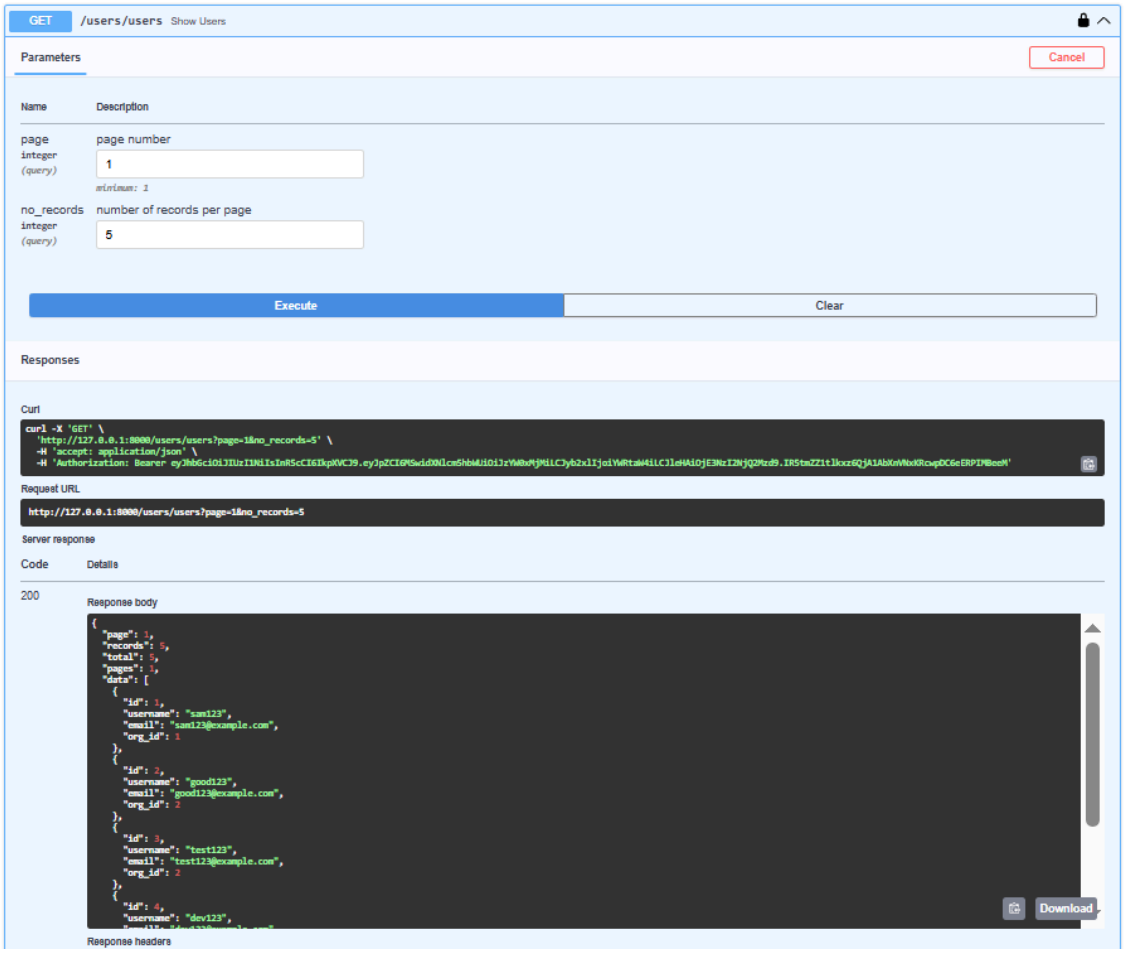
```python
from pydantic import BaseModel, EmailStr, Field
from typing import Optional,List,Generic,TypeVar
from schemas.user_schema import Userout

resources=TypeVar("resources")

class Page(BaseModel, Generic[resources]):
    page :int
    records: int
    total: int # Total number of records e.g user
    pages: int  # Total number of pages
    data: List[resources]
```

User_routes and handler

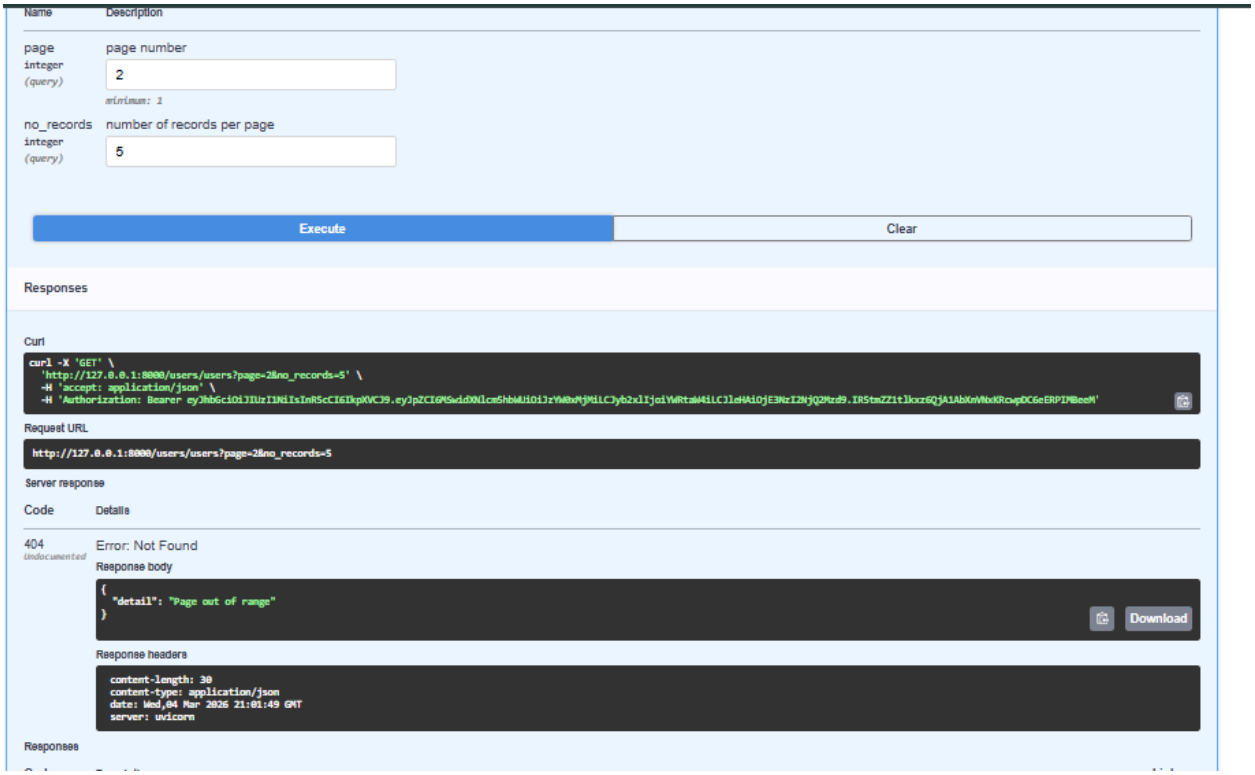
```python
#for admin to see all users
@user_routes.get("/users",response_model=Page[Userout])
def show_users(db: Session = Depends(get_db), current_user: User =
Depends(role_required("admin")),
               page: int = Query(1, ge=1,description="page number"),
               no_records: int = Query(5, description="number of records
per page")

               ):

    return show_all_users(db, page, no_records)
```

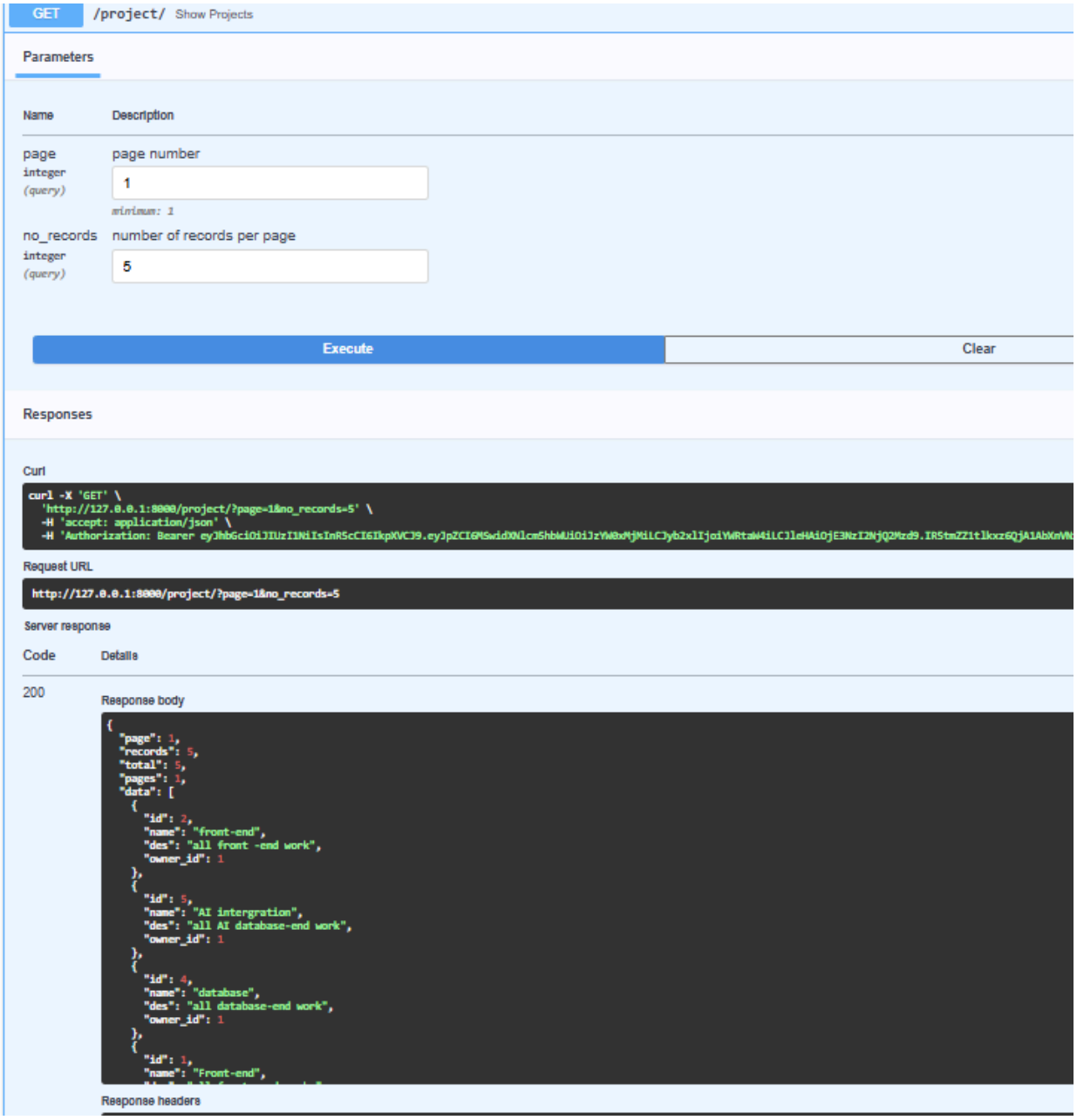
```python
def show_all_users(db: Session, page: int, no_records: int):
    users = db.query(User).all()
```

```
    total = len(users)


    start = (page - 1) * no_records

    if start >= total:
            raise HTTPException(status_code=404, detail="Page out of
range")
    end = start + no_records


    # Slice the data
    if total>=no_records:

        paginated_data = users[start:end]
    else:
         paginated_data = users


    pages = (total + no_records - 1) // no_records

    if page > pages:
        raise HTTPException(status_code=404, detail="Page out of range")

    return {
        "page": page,
        "records": no_records,
        "total": total,
        "pages": pages,
        "data": paginated_data
    }
```



Similarly for others


Name
Description
page
integer
(query)
page number
2
minimum: 1
no_records
integer
(query)
number of records per page
5
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://127.0.0.1:8000/users/users?page=2&no_records=5' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidXNlcm5hbWUiOiJrYW0xMjMiLCJyb2xlIjoiYWRtaW4iLCJleHAiOjE3NzI2NjQ2Mzd9.IR5tmZZ1tlkxrzGQjA1AbXnVNxKRcwpDC6eERPIMBeeM'
Request URL
http://127.0.0.1:8000/users/users?page=2&no_records=5
Server response
Code
Details
404
Undocumented
Error: Not Found
Response body
{
"detail": "Page out of range"
}
Download
Response headers
content-length: 30
content-type: application/json
date: Wed,04 Mar 2026 21:01:49 GMT
server: uvicorn
Responses


GET
/project/ Show Projects
Parameters
Name
Description
page
integer
(query)
page number
1
minimum: 1
no_records
integer
(query)
number of records per page
5
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://127.0.0.1:8000/project/?page=1&no_records=5' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidXNlcm5hbWUiOiJzYW0xMjMiLCJyb2xlIjoiYWRtaW4iLCJleHAiOjE3NzI2NjQ2Mzd9.IR5tmZZ1tlkxz6QjA1AbXnVW
Request URL
http://127.0.0.1:8000/project/?page=1&no_records=5
Server response
Code
Details
200
Response body
{
"page": 1,
"records": 5,
"total": 5,
"pages": 1,
"data": [
{
"id": 2,
"name": "front-end",
"des": "all front -end work",
"owner_id": 1
},
{
"id": 5,
"name": "AI intergration",
"des": "all AI database-end work",
"owner_id": 1
},
{
"id": 4,
"name": "database",
"des": "all database-end work",
"owner_id": 1
},
{
"id": 1,
"name": "Front-end",
Response headers